Narrowcast Transaction Relay

Adaptive Packaging for Bitcoin Transaction Propagation

Eric Voskuil
eric@voskuil.org

Phillip Mienk
mienkphi@gmail.com

August 2022

Abstract

Bitcoin transaction relay operates as a fee-filtered broadcast, a design that necessarily produces orphan transactions, strands valid transactions behind peer relay policy, and complicates fee bumping. Recent proposals address these symptoms through wallet-constructed transaction packages, which neither eliminate orphans nor resolve the deficiencies of the `mempool` message. This paper surveys the formal proposals pertaining to transaction relay, analyzes the structural causes of orphaning, stranding, and pinning, and proposes a change from fee-filtered broadcast to true *narrowcast* relay. Under narrowcast relay, packages are constructed and deconstructed by nodes—not wallets—using per-channel knowledge of peer fee policy. A package is never a precomputed artifact but a per-channel, moment-in-time announcement grouping; origination is not a special case, as a wallet is a client of its node over a private channel, not a peer. A single new inventory type identifier (`MSG_PACKAGE`) suffices; no other protocol change is required, and a silent implementation requiring no wire change at all is possible. The result is a relay network in which orphans are never received—closing a demonstrated topology-inference vector along with them—child-pays-for-parent fee bumping and replacement by conflict are inherently enabled, and the `mempool` message is repaired.

1 Introduction

Transaction relay in Bitcoin has accumulated a substantial body of formal proposals. Those pertaining directly to relay are:

- BIP035 (`mempool` message)
- BIP037 (connection bloom filtering)
- BIP125 (replace-by-fee signaling)
- BIP133 (`feefilter`)
- BIP144 (segwit peer services)
- BIP152 (compact block relay)
- BIP156 (Dandelion — rejected)
- BIP157 (client-side block filtering)
- BIP330 (transaction announcements reconciliation — Erlay)

- BIP338 (`disabletx`)
- BIP339 (wtxid-based transaction relay)

These proposals may be organized by the aspect of relay they affect: transaction *form*, transaction *pull*, and transaction *push*. This taxonomy clarifies which proposals bear on the problem addressed here.

1.1 Transaction Form

BIP125, BIP144, and BIP339 pertain to the form of a transaction or its identifier. BIP125 attaches "replacement" policy semantics to the `tx.sequence` field, while also specifying a necessary fee increment over any "replaced" transaction(s). BIP144 and BIP339 pertain only to the serialization of a transaction or its unique identifier.

1.2 Transaction Pull

BIP035, BIP152, and BIP157 are pull mechanisms. BIP035 introduces the `mempool` message, a request for "all the transactions in the node's transaction memory pool," with an acknowledgement that the response will be dropped if it exceeds 50,000 entries. The others are requests for specifically-identified transactions.

1.3 Transaction Push

BIP037, BIP133, BIP156, BIP330, and BIP338 affect push behavior.

BIP037 introduces a request for narrowing to a bloom-filtered subset of transactions, while also introducing a handshake signal to disable transaction relay until receipt of a new filter message. It is being phased out due to privacy leakage, with BIP157 as an alternative. BIP338 refines the handshake signal of BIP037 to allow a node to permanently disable transaction relay for a connection, while also recommending disabling of address relay; this facilitates blocks-only nodes. BIP156 was an attempt to obscure the identity of a transaction's originating node; it was rejected but is included for completeness.

BIP330 introduces a complex protocol for reducing transaction relay network traffic. Each node maintains a reconciliation state for each peer, deferring transaction broadcast based on a two-second timer in order across its set of outbound-connected peers. Batching reduces frequency and enables more efficient reconciliation, which reduces traffic. It also allows for transaction flooding to "well-connected publicly reachable nodes." Transaction relay latency is 183% of Bitcoin's flooding protocol. As the proposal describes:

> Every node stores a set of 128-bit truncated IDs for every peer which supports transaction reconciliation, representing the transactions which would have been sent according to the regular flooding protocol. Incoming transactions are added to sets when those transactions are received (if they satisfy the policies such as minimum fee set by a peer).

BIP133 introduces the `feefilter` message, a request for narrowing to a fee-filtered subset of transactions, applied to announcements as well as BIP035's `mempool` message and BIP037's bloom filtering. This is the proposal of primary interest here. Per BIP133:

- the node will be permitted, but not required, to filter transaction `inv`s for transactions that fall below the feerate provided in the `feefilter` message;
- the fee filter is additive with a bloom filter for transactions;
- `inv`s generated from a `mempool` message are also subject to a fee filter.

2 Related Work

Packaging as a relay mechanism was raised in December 2018 [13], as an enumeration of design questions rather than a specification. The first question asked is whether the sender or the recipient should initiate, and the enumeration favours recipient initiation on the ground that a sender announcing the ancestors of each transaction would cause wasteful repeated announcement across every link. That objection holds against a sender that announces ancestry indiscriminately. It does not hold against a sender that records what it has already sent on each channel, which is the marking of Section 4.2.2 and costs one per-channel record per pool entry. The enumeration does not consider a sender that consults the recipient's advertised fee rate, and does not address orphan handling. Sender-side relay was therefore weighed and set aside on a cost that per-channel state removes, rather than on the question this paper takes up.

A "Generic Package Relay" proposal was posted to the bitcoin-dev mailing list on May 17, 2022 [14], and was subsequently formalized as BIP331, Ancestor Package Relay [10]. Its abstract reads:

> Enable the use case in which a child pays for its otherwise-too-low-fee parent(s) and their mempool conflict(s). Handle orphan transactions more reliably and reduce txid-based relay.

Its motivation is summarized as follows. Incentive-compatible mempool and miner policies help create a fair, fee-based market for block space. While miners maximize transaction fees in order to earn higher block rewards, non-mining users participating in transaction relay reap many benefits from employing policies that result in a mempool with the same contents, including faster compact block relay and more accurate fee estimation. Additionally, users may take advantage of mempool and miner policy to bump the priority of their transactions by attaching high-fee descendants (child pays for parent, or CPFP).

Only considering transactions one at a time for submission to the mempool creates a limitation in the node's ability to determine which transactions have the highest feerates, since it cannot take into account descendants until all the transactions are in the mempool. Similarly, it cannot use a transaction's descendants when considering which of two conflicting transactions to keep (replace by fee, or RBF). When a user's transaction does not meet a mempool's minimum feerate and they cannot create a replacement transaction directly, their transaction will simply be rejected by this mempool. They also cannot attach a descendant to pay for replacing a conflicting transaction.

This limitation harms users' ability to fee-bump their transactions. Further, it presents a security issue in contracting protocols which rely on presigned, time-sensitive transactions to prevent cheating (HTLC-timeout in LN penalty, unvault cancel in Revault, refund transaction in discreet log contracts, update in eltoo). In other words, a key security assumption of many contracting protocols is that all parties can propagate and confirm transactions in a timely manner. Increasing attention has been brought to "pinning attacks," a type of censorship in

which the attacker uses mempool policy restrictions to prevent a transaction from being relayed or getting mined.

These transactions must meet a certain confirmation target to be effective, but their feerates are negotiated well ahead of broadcast time. If the forecasted feerate was too low and no fee-bumping options are available, attackers can steal money from their counterparties. Always overestimating fees may sidestep this issue temporarily (while mempool traffic is low and predictable), but this solution is not foolproof and wastes users' money. For some attacks, the available defenses require nodes to have a bird's-eye view of Bitcoin nodes' mempools, which is an unreasonable security requirement.

The proposal concludes that the best solution is to enable nodes to consider packages of transactions as a unit—e.g., one or more low-fee parent transactions with a high-fee child—instead of separately. A package-aware mempool policy can help determine if it would actually be economically rational to accept a transaction to the mempool if it does not meet fee requirements individually.

A single use case of txid-based relay remains: handling "orphans," transactions spending output(s) from a transaction the node is unaware of. Orphans are common for new nodes that have just completed initial block download (IBD) and do not have an up-to-date mempool. Nodes may handle orphans by storing them in a cache and requesting any missing parent(s) by txid (inputs specify by txid, not wtxid). These parents may end up being orphans as well, if they also spend unconfirmed inputs that the node is unaware of. This method of handling orphans is problematic for two reasons: it requires nodes to allocate memory for unvalidated data received on the P2P network, and it requires txid-based relay between two wtxid-relay peers. The proposal aims to make orphan-fetching more efficient.

2.1 Critique of Packaging Proposals

Such packaging proposals share four deficiencies. They:

- do not solve the core problem;
- do not eliminate orphans;
- do not fix the `mempool` message;
- rebroadcast all ancestor txids in order to bump them.

2.2 Discussion

The proposal was discussed at length on the mailing list through May and June 2022, where the present authors' critiques were first posted; the narrowcast design specified in Section 4 was posted June 8, 2022, forking the discussion into its own thread [15]. Three exchanges from that discussion bear directly on the present design.

First, against the accumulation of announcement metadata. Aggregate fee and weight fields, per-transaction fee and size lists, and ultimately a serialized topology graph were each proposed as additions to the package announcement, so that a receiver might judge a package before downloading it. But the set of transactions *is* the graph—and is likewise the fee, the size, and the sigops. Announced metadata cannot be validated without downloading the transaction set it describes, so it can never safely drive a decision; the only information packaging adds is the *association* of transactions for aggregate rather than individual consideration. A rational package is a block, or compact block, without the header.

Second, fee advertisement is wrong in principle. Single-transaction relay carries no advertised feerate; BIP133's `feefilter` already provides the required semantics and should be interpreted as applying to packages, with the same recourse against violation—dropping the peer. This argument was accepted by the package relay proposal's author, and aggregate fee and weight were removed from its package information message on June 7, 2022.

Third, on wholesale reuse of BIP152. A package being a compact block without a header suggests reusing the compact block messages outright, with a Merkle root over the partially ordered transaction set standing in for the absent header as package identity. This was rebutted on denial-of-service grounds: compact-block short identifiers are computed against the whole mempool under a per-connection salt, and packages, carrying no proof-of-work gating, arrive orders of magnitude more frequently than blocks; short-identifier collisions are also cheap enough to manufacture that using them for deduplication creates a censorship vector. The rebuttal stands. The present proposal accordingly defines no package identity at all—no hashing and no negotiation beyond a version—narrowing the wire change to the single inventory type of Section 4.

2.3 Prior Art

The mechanisms this proposal composes are, with one exception, established practice. Stating their provenance clarifies what is and is not new.

Per-peer knowledge state. Marking is not a new requirement imposed on nodes. Bitcoin implementations already maintain per-peer inventory knowledge to suppress echo and to validate requests; an exact per-peer set was replaced years ago by a rolling Bloom filter for bounded memory. What is new here is the *use* of that state: it becomes an input to what the sender constructs, rather than only a filter on what it repeats. This changes a correctness requirement. A probabilistic filter admits false positives, which under echo suppression merely omit a redundant announcement, but under narrowcast would cause the sender to believe a peer holds an ancestor it does not—silently stranding the descendant. Marks must therefore be exact, and their storage bounded by pool lifetime (Section efsec:incoming) rather than by a decay window.

Receiver-tailored minimal transfer. Computing a minimal transfer against an advertised or inferred receiver state is standard outside Bitcoin. The Git pack protocol negotiates possessed and wanted objects and transmits a pack computed for that particular client; NNTP's offer and request exchange [23] carries per-peer article state; content-based publish–subscribe systems filter per subscriber, and the term *narrowcast* is drawn from that literature [22]. Within Bitcoin, BIP152 already constructs a per-peer message against assumed receiver state. The present proposal applies the same discipline to transaction relay.

Dependency-ordered delivery. The invariant that an orphan is never received is causal delivery, long studied in distributed systems [19, 21]: a message is delivered only once the messages it depends upon have been. Such systems resolve out-of-order arrival in one of two places—buffering at the receiver until dependencies arrive, or enforcing order at the sender. Bitcoin has chosen the former, and an orphan pool is that buffer. The argument of Section efsec:pushpull is that receiver buffering is untenable against an adversary—and not because the buffer is unbounded. It must be bounded, precisely to cap the memory that an unvalidated stranger's data may occupy. It is the bound that defeats it: any bound is trivially churned by a peer that floods it, so an entry's survival until its ancestor arrives is never assured, and retention can never be relied upon as a matter of protocol. Sender enforcement moves the guarantee to the side that can supply it without holding state on another's behalf.

Epidemic dissemination. Gossip with per-peer state descends from the epidemic algorithms of Demers et al. [20], whose anti-entropy branch leads to the set reconciliation of BIP330.

What is not found in this prior art is the criterion. Bundling by transitive dependency is old; bundling so that the aggregate clears the *receiver's advertised price*, recomputed per link and carrying no transaction unnecessary to that threshold, appears to be without precedent. The contribution claimed here is therefore narrow: not per-peer state, not tailored construction, and not ordered delivery, but the use of an economic threshold as the packaging criterion, and the elimination of orphan receipt that follows from it.

Note Added in Proof (August 2026)

The body of this paper was written in August 2022 and analyzes the package relay proposal as posted; the discussion above reflects the mailing list record through June 2022.

The narrowcast thread [15] was revived in September–October 2022. Its substantive critique came from Suhas Daftuar (October 4): sender-side packaging requires per-peer relay state, since fee filters are dynamic and knowing whether a peer received a past ancestor requires caching relay history; and feerate is ill-defined over subgraphs—mempools legitimately diverge by arrival order under BIP125's fee and feerate requirements—so a sender cannot predict peer state, and receiver-side dependency retrieval remains necessary. The authors' response: the per-peer state is precisely the marking of Section efsec:process, order effects are absorbed by channel bookkeeping (a node knows what it has relayed on each channel), and sufficiency holds to the extent that policies are published.

The first limb of that critique is answered by the deployed code rather than by argument. Per-peer relay state was not a new requirement in 2022; implementations already carried it, as Section 2.3 sets out, and had done so for years—the objection amounts to asking a node to consult a record it is already keeping. What this proposal changes is not whether the state exists but whether it may be trusted, since a filter adequate for suppressing echo is not adequate for deciding what to construct. The cost at issue was therefore the difference between an approximate record and an exact one, not the difference between no record and one. The exchange ended unresolved on the second limb: whether unpublished policy is a fact of the network to be designed for, or a self-inflicted orphaning to be starved out. The dynamic-feefilter component is formalized in the Mark Staleness analysis (Section 5.9).

The package relay proposal was formalized as BIP331 (Ancestor Package Relay), created August 8, 2022, where it remains in draft status. Its November 2022 revision moved substantially toward the simplifications pressed in the discussion above: sender-initiated announcement was dropped (deferred pending deployment data), the child-with-parents package type was dropped in favor of ancestor packages, and the chain-tip (blockhash) coupling was removed, chainstate differences being handled inside mempool validation instead—this in addition to the June removal of fee and weight metadata. Deployment has proceeded piecemeal: Bitcoin Core v28.0 (October 2024) shipped "opportunistic" one-parent-one-child package relay, in which a node retains a below-minimum-fee transaction in a bounded orphan cache and re-evaluates it as a package upon later arrival of a fee-paying child; the mechanism is explicitly best-effort, propagating packages in the non-adversarial case only. Subsequent releases refined the mechanism (v30.0) and relaxed its parent constraints (v31.0). The shipped design thus adopts receiver-side package evaluation while retaining bounded orphan retention and txid-based fetching—consistent with the trajectory critiqued in this section, and distinct from the sender-side narrowcast construction proposed here, which eliminates orphan receipt altogether.

That outcome was predicted publicly, on the record, before any of it shipped. The narrowcast proposal was posted to bitcoin-dev in June 2022 [15] on the ground that a node complying with fee policy produces *predictable* orphans, and that a correct design must therefore never announce one; the same thread pressed the objection against package relay that October, holding that the approach would continue to disseminate and chase orphans, and that any reliance on orphan retention is a denial-of-service allowance rather than a mechanism. Sections 3.1 and 3.8 of this paper, written contemporaneously, state that argument in full: an orphan pool can be trivially overflowed, so retention can never be relied upon as a matter of protocol, and a node that drops an orphan and chases its ancestors never gains ground on the announced lineage. Both have since been observed on mainnet, and deployment has exceeded the objection in degree.

The shipped mechanism holds the child in the orphan pool, unprotected, while it fetches the parent—so its propagation depends on exactly the retention this analysis denied it. Node instrumentation reported in 2025 [16] recorded removal of more than one hundred thousand orphans per minute on mainnet in September 2024, which turns a hundred-entry pool over on the order of a thousand times a minute, leaving an entry's expected survival in the tens of milliseconds. The pool was characterized in the same report as trivially churnable by any single peer, and the observed rate was thought possibly to be such an attack already under way. Both may be true at once, and the report cannot say which—nor could any report, because that determination is not available to a node under fee-filtered broadcast. An orphan there is produced by compliant and by hostile behavior alike, and the two are indistinguishable at the receiver, so no accounting taken over orphans can be principled: the quantity being bounded cannot be attributed. The stated consequence was that "opportunistic 1p1c is useless in the presence of adversaries." Nodes were further observed forgetting orphans and re-requesting them repeatedly, which is the chase of Section 3.8 in the field. The global limit was accordingly replaced with per-peer denial-of-service accounting in version 30.0. The remedy of simply enlarging the pool—proposed on measurement grounds in 2019, where a tenfold increase was calculated to render the overhead negligible [18]—was by then explicitly rejected, on the ground that raising the limit does not address churnability at all.

That indistinguishability is the deeper defect, and it is what Section 5.10 removes. Where a compliant node never announces an orphan, the appearance of one is a protocol violation rather than an ambiguity—attributable to the peer that sent it, and answerable by dropping that peer. The measurement above is the cost of the alternative: a figure that cannot be classified, gathered in defense of a mechanism whose soundness depends on the classification. The bound was not the flaw to be tuned away; reliance on retention was.

A draft of the per-channel packaging algorithm was published contemporaneously with the proposal [24]; a corrected revision (August 2026) [25] appears as Appendix A.

3 Structural Analysis of Relay

3.1 Push and Pull

Each protocol has a client and a server aspect. Where both are implemented on both ends of a channel, the nodes are *peers*. A node is both a query server and a subscription service, and a node is also a query client and service subscriber.

Queries are arbitrary; a peer may essentially request whatever it wants within protocol structural constraints:

- range of headers
- individual blocks
- individual transactions
- mempool identifiers
- addresses

Maintaining a quality-of-service guarantee by the server is therefore fundamentally a function of rate-limiting requests. A client should be expected to drop a server that provides invalid responses, including:

- disconnected header
- invalid block
- invalid transaction
- malformed address

A broadcast service does not consider the state of its subscribers. Bitcoin nodes have operated largely in this manner. Broadcasting includes block announcement (by identifier, header, or compact form) and transaction announcement (by identifier).

A broadcast object that does not connect to the client's blockchain or transaction pool is referred to as an *orphan*—it has at least one missing ancestor. Any requirement to retain orphans opens a node to denial-of-service attacks. Nodes may have retained orphan pools of blocks and/or transactions historically, but this is not a tenable architecture in a hostile environment: an orphan pool can be trivially overflowed. It must never be the case that a node must retain an orphan as a matter of protocol.

Yet as a broadcast service, a node will produce orphans simply by complying with its own and/or its peer's fee policy. By not sending below-peer-feerate transactions, and later sending above-peer-feerate transactions, an orphan is received by the peer. By not accepting below-feerate transactions, one must then expect to receive orphans.

This is observable in the field. A measurement campaign on live nodes found that the missing parents of orphan transactions exhibit systematically lower fees and larger sizes—hence lower fee per byte—than transactions generally, and that orphan handling imposed a network overhead exceeding seventeen percent at the default pool size [18]. The correlation is reported there without a causal account. It is the signature this mechanism predicts: the orphan population is the residue of fee filtering, and the parents found missing are the transactions the filter withheld. Orphaning is thus not an incidental defect of implementations but the arithmetic consequence of relaying under an advertised price.

3.2 Stranding

A valid transaction may fail to relay as a result of failure to satisfy peer relay policy. It must be assumed that policy may vary widely. As there is no way to know the relay policy of a peer-to-peer network, stranding is common.

3.3 Bumping

Fee bumping is increasing net fee in order to free stranded transactions. The two approaches are CPFP (child pays for parent) and RBF (replace by fee). In CPFP a descendent transaction is issued. In RBF a conflicting transaction is issued with a fee sufficient to "replace" its set of conflicting transactions (generally limited to unconfirmed transactions).

3.4 Pinning

Allowing RBF exposes a node to a denial-of-service attack, as any number of valid yet conflicting transactions may be received. The attacker pays for only one of them, while the node pays for storage of all of them. As such, the minimum fee for any group of conflicting transaction sets is the minimum fee of the group as if it would be mined as a whole. This necessary protection exposes transactions to *pinning*, which can significantly raise the cost of bumping. CPFP does not "replace" transactions, so it is not subject to this attack. However, other arbitrary "packaging" policies can expose it to pinning, and have resulted in additional complexities.

3.5 Policy

Employment of any standardness rule has the same effect as minimum fee policy: it produces orphans.

3.6 Packaging

Packaging is an idea that has been around for some time. The basic idea is to group a set of transactions for network relay, where each node considers the package in the aggregate. This allows a stranded transaction or set of transactions to be reintroduced to the network with a fee bumped by CPFP. From a relay perspective, this is comparable to issuance of a single transaction. The package is an atomic unit, with a size, a set of sigops, and a fee. For a node that has not previously accepted any of its transactions, it would effectively be just a larger transaction. The package must be announced, downloaded (possibly optimized with differential downloading, similar to compact blocks), validated, stored, and relayed. It remains subject to all of the existing problems while adding complexity to wallets, which must produce the packages. This approach kicks the can down the road, not solving the inherent problem of broadcast relay.

3.7 BIP330/Erlay

The objective of the present proposal is to enable efficient bumping of stranded transactions while eliminating the relay of orphans. BIP330 is a proposal to reduce network traffic by batching and differencing; it does not deal with either of these questions. The question remains *what set of transactions to relay*, as it is this set that BIP330 operates over.

3.8 Ancestor Chasing

A node receiving an orphan may request the transactions that it believes it is missing from the sending peer. It may buffer the orphan while awaiting these responses, but it must not rely on that buffer: it cannot tell how long the peer will take to respond (if ever), and it cannot validate the orphan in the meantime. Any such buffer must be bounded, and a bound the node is not prepared to see exhausted is not a bound at all, so no protocol behavior may be predicated on

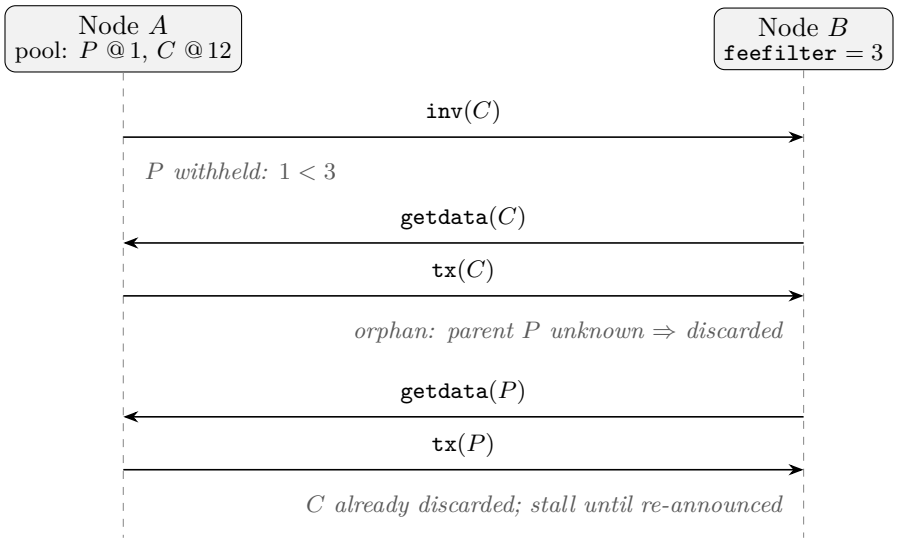


Figure 1: Orphan production under fee-filtered broadcast, and the ensuing chase. Node A complies with B's advertised fee rate by withholding the below-rate parent P while announcing the above-rate child C; the orphan is thereby produced by policy compliance, not by misbehavior. B cannot retain C while awaiting the ancestor, so on receipt of P it has no descendant to attach and gains no ground—the "wild goose chase."

an orphan's survival. A malicious peer can otherwise put the node into a perpetual cycle of orphan accumulation and chasing of bogus endpoints.

At best the node drops the orphan and requests the missing ancestor(s). Then, upon receipt of ancestors, it does so again, until eventually receiving (or not) a transaction that it can validate in its context. Yet at that point it has dropped all descendent transactions and has no way to recover them. It therefore stalls on this transaction until a new descendent arrives, at which point it goes through the same cycle, adding one more transaction to the previous. Note that it never gets any closer to the announced transactions in this lineage. We refer to this as the "wild goose chase."

The packaging proposal of Section efsec:related aims to make "orphan fetching more efficient." The present proposal eliminates orphan dissemination altogether. Figure 1 traces both halves of the failure: the orphan is produced by policy compliance alone, and the chase that follows never converges.

4 Narrowcast Transaction Relay

In order to fix transaction relay, it can be changed from a fee-filtered broadcast to a true *narrowcast*. In narrowcast transaction relay we start from the assumption that peers have a synchronized view of the confirmed transactions, and that a node's new peer has an empty transaction pool (context).

We define a new `inv` type identifier, `MSG_PACKAGE`,[1] that identifies associated transactions by transaction ID (type determined by the `wtxidrelay` message), referred to collectively as a *package*. Nothing more is required of the protocol. The following relevant scenarios exist:

[1] Named `MSG_TX_PACKAGE` in the June 2022 posting [15]; shortened here.

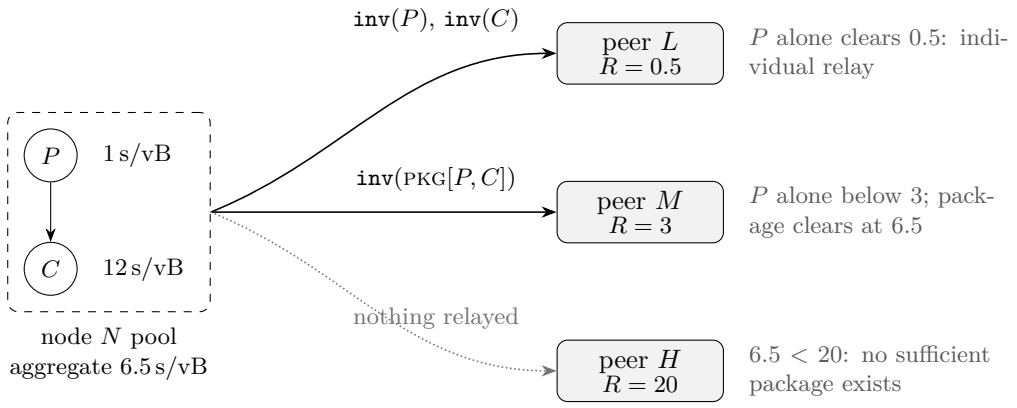


Figure 2: Narrowcast relay of one accepted transaction across three channels. The same pool state yields a different announcement on each link, determined solely by the peer's advertised fee rate: individual transactions where the parent already clears, a package where only the aggregate clears, and silence where even the aggregate does not. A package is therefore not an artifact that propagates but a per-channel construction that dissolves at the next hop.

- receipt of an `inv` containing `MSG_TX` and/or `MSG_WTX` identifier(s);
- receipt of an `inv` containing `MSG_PACKAGE` identifier(s).

4.1 Packages

Packages are constructed and deconstructed by *nodes*, not wallets. Only nodes are aware of the minimum fee rates of their peers.

4.1.1 Definitions

Each transaction announced by `MSG_TX`/`MSG_WTX` is a single-element package. N transactions announced by `MSG_PACKAGE` in a single `inv` are an N-element package.

orphan package
: is missing at least one ancestor in the given context

invalid package
: does not satisfy consensus in the given context

excessive package
: exceeds single-block scope

redundant package
: contains a transaction currently marked on the channel

insufficient package
: does not satisfy policy in the given context

unconfirmable package
: conflicts with at least one confirmed transaction in the given context

acceptable package
: none of the above

conflicting package
: acceptable package that conflicts with at least one unconfirmed transaction in the given context

4.1.2 Constraints

Package constraints are the same as for individual announcements, with an additional orphan exclusion:

- a package must be valid within the context of a single block;
- a package must be minimally sufficient for the recipient minimum fee rate;
- a package must be partially ordered in the manner of block transaction ordering;
- a package must not be an orphan, but it may be conflicting.

The ordering constraint serves fail-fast validation. In blocks, transaction ordering is an implementation artifact that leaked into consensus, where it needlessly constrains validation parallelism; in packages, which carry no proof-of-work gating, the partial order allows the receiver to attach and validate incrementally, dropping the attempt—and, at its discretion, the peer—at the first orphan encountered. Per-message work is thereby kept at the level of ordinary transaction relay.

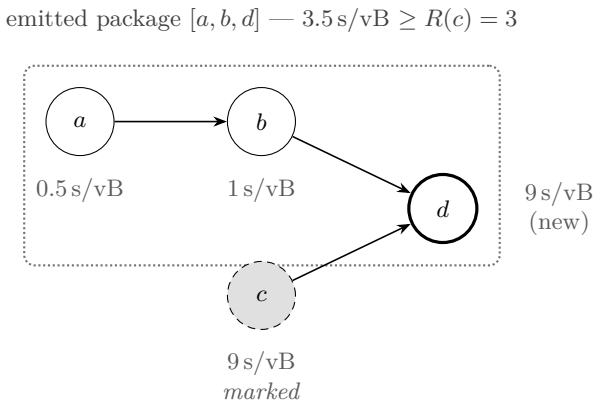
4.2 Process

4.2.1 Incoming

A node receives a package and iterates through each transaction in order, attaching inputs to existing transaction pool or blockchain outputs and validating each transaction, until the package is fully validated. If any previous output is not found, the package is an *orphan*. If any transaction does not validate, the package is *invalid*. If the size or sigops of the package cannot fit within a block, the package is *excessive*. If any transaction is currently marked on the channel, the package is *redundant* (redundancy is thereby verified in constant time against the "marks" for the peer). If the sum of fees divided by the sum of bytes is below the node's fee minimum, the package is *insufficient*. If the package spends any confirmed output, it is *unconfirmable*. If the package spends any output spent in the transaction pool, it is *conflicting*. Otherwise the package is *acceptable* (which includes conflicting).

Note that a conflicting package must have satisfied conflicting-package fee policy in order to not be insufficient. An acceptable package is added to the transaction pool. A conflicting package may be added to the transaction pool and, depending on implementation, may "eject" from the transaction pool the package with which it conflicts.

Each transaction identifier received from a peer may be marked within the node's transaction pool as *known* to that peer. A mark records only that the peer holds the transaction, and is set alike by sending it and by accepting it from that peer; the two directions are equivalent for relay purposes, since either establishes the fact the sender needs. This provides a normalized mechanism for eliminating package echo and for verifying non-redundancy. Marks share the lifetime of the pool entry they annotate: a transaction that leaves the pool—confirmed, evicted,



Walk from d: input c is marked, so the traversal prunes there; inputs b and a are unmarked and unconfirmed, so both are collected. Postorder emission yields $[a, b, d]$—every in-package ancestor precedes its descendant. Neither a (0.5) nor $a + b$ (0.75) clears the channel rate, so the first sufficiency crossing occurs at d, and the package is prefix-minimal.

Figure 3: The outgoing walk over a pool subgraph for a channel at $R(c) = 3$. Marks prune the traversal, bounding search to the unconfirmed ancestry the peer is not already known to hold; the previously relayed c is neither rewalked nor re-announced.

or ejected—sheds its marks on every channel. Discard is symmetric forgetting, so a discarded transaction may traverse the channel again within a future package, as when an ejected lineage is revived as a conflicting package. Redundancy is thus a statement about present pool state, not channel history; a channel-history rule would require unbounded per-channel memory, which the protocol must never require.

4.2.2 Outgoing

The accepting channel notifies all channels of each attached transaction individually, in the partial order of the package. Each channel then builds an outgoing package for its peer. This is where packages are created.

Given a single accepted transaction, existing blockchain and transaction pool context, and the feerate of the peer, a channel attempts to build a minimally sufficient package for its peer. To do this it performs a linear walk of the unconfirmed (and unsent) ancestry of the transaction, announcing minimally sufficient packages to the peer in reverse depth order. Each announced transaction is marked known for the channel. Ancestors of marked transactions need not be marked, and any such preexisting marks can be purged.

If no minimally sufficient package can be found within block constraints, nothing is relayed. This implies a large set of insufficient-fee transaction ancestors to the package (or within it). While there may be a larger sufficient package, it cannot be mined by the peer to recover a sufficient fee (and the peer cannot assume recovery via the subsequent block). In this case the peer will not receive subsequent descendent transactions until the low-fee ancestors are confirmed, as the excessive constraint will continue to prohibit their packaging until that point.

5 Analysis

5.1 Storage and Retrieval

There is a storage/memory cost to marking, where a mark must be retrieved based on transaction identifier. This can be implemented as a hash table of channel identifiers per pool transaction. Channel identifiers may be purged as channels are closed (or through subsequent garbage collection).

5.2 Complexity

A fully synchronized peer implies one mark for each leaf of the transaction pool. The maximal marking is for a completely flat and fully synchronized pool—one mark per pool entry. The maximal search space is the full transaction pool. Search cost is $O(N)$ where N is the unconfirmed (and unmarked) ancestry of the transaction. The worst-case search space is the full transaction pool, in the case where an announced transaction depends on every transaction in the pool; however, this is reduced to the space of leaf nodes once the pool is marked. Marking is always $O(1)$; given hash-table marking, retrieval is $O(1)$ and purge is $O(N)$ in the size of the transaction pool.

5.3 State

This does not necessarily produce transaction pool synchronization, as only transactions pertaining to announced packages are potentially relayed. However, as transactions are confirmed and new transactions relayed, peer transaction pools converge (eventual consistency) to the extent they have the same feerate.

5.4 Latency

Relay latency is increased by search cost, yet once connections are mature this is reduced to zero in the case of a single-element package. In the case of multiple-element packages, behavior is improved, as this would previously have resulted in the relay of an orphan. Given that maturity can be no greater than the time required to clear the transaction pool that existed at the time of connection, it can reasonably be assumed that there is always a low-latency path to any long-running node. Individual packages are bounded in size/sigops at the same level as individual transactions.

5.5 Redundancy

If a peer connects with a non-empty transaction pool, it should expect to receive transactions it already holds. These are not *redundant packages* in the sense of Section 4.1.1: a new channel carries no marks, so nothing announced on it is marked, and such a package is acceptable and will be processed as one. It is merely duplicative, and the bandwidth spent on it is wasted. Once marks have accumulated there should be no duplication from a peer, except in the case of a race between a package obtained from peer A which is at the same time received and relayed from peer B before it can be relayed to peer B. This proposal does not attempt to reduce such duplication (see BIP330).

Initial-connection duplication is increased due to the presumption of an empty peer transaction pool. This is the purpose of the `mempool` message (see BIP035), which may be used by

both peers upon connection to bootstrap mempool state. Given size constraints on the message payload (unaddressed by BIP035), this can be problematic. Furthermore, neither BIP035 nor BIP133 (`feefilter`) addresses coherency or sufficiency of the `mempool` message payload. For these reasons this proposal refines the `mempool` message (Section 6).

5.6 Origination

A transaction originates with a wallet, yet on the peer-to-peer network there are no wallets; there are only nodes. A wallet that appears to join the network is not a peer—it is a client, and its trusted node is the peer. The wallet's connection is a private channel to its own node. In practice this may be a light wallet connected to a node that accepts the wallet's transactions under a zero fee rate policy—specifically because the node trusts the wallet.

This resolves the question of who constructs a package at origination. One might suppose that a wallet bumping a stranded transaction must construct a package, since a downstream peer may lack the transaction's unconfirmed ancestry: the ancestry may never have been accepted (below the peer's floor at submission), may have been evicted under mempool pressure, may have been ejected by a conflicting package, or the channel may be new (the empty-context presumption following reconnection). In every such case, sending the bumping descendant alone would produce an orphan at the first hop, and orphans are dropped.

However, because the client can always inject its transactions into its own node regardless of fee, stranding can never mean that a transaction exists only in the wallet. The origination node always holds the full unconfirmed ancestry, and when a fee-bumping descendant arrives, the standard outgoing channel walk (Section 4.2.2) constructs the package. Each of the ancestry-absence cases above is handled identically by the same algorithm. Origination is not a special case of the protocol, and a package is never a precomputed artifact: it is a per-channel, moment-in-time announcement grouping, a function of the peer's current fee rate, the peer's context, and the sender's unconfirmed ancestry. This is the essential contrast with wallet-authored packaging proposals (Section efsec:related), which push a static application-layer object through the network intact.

5.7 Policy and Denial of Service

Zero-fee acceptance on the private channel is not an absence of policy but a policy setting: policy is a per-channel mechanism, and the fee rate is one of its parameters. Any policy setting must be sustained by denial-of-service protection commensurate with what it exposes; zero-fee acceptance from an arbitrary counterparty is an unbounded grant of free storage and relay. Trust is the ordinary protection for a private channel, but it is not strictly required—zero-fee service can be offered publicly under some other protection scheme (such as rate limiting, proof of work, payment, or identity). Every mechanism of this proposal on the public channel—fee filtering, the prohibition on orphan retention, redundancy marks, the conflicting-group fee floor—is likewise a denial-of-service protection warranted by an untrusted counterparty. All channel distinctions therefore reduce to this one distinction: the protection scheme under which the channel's policy is sustainable, with the private channel to one's own node as the trusting case.

5.8 Reciprocity and the Server Layer

As established in Section 3.1, nodes are peers where both the client and server aspects of the protocol are implemented on both ends of a channel. This yields a criterion for what belongs in

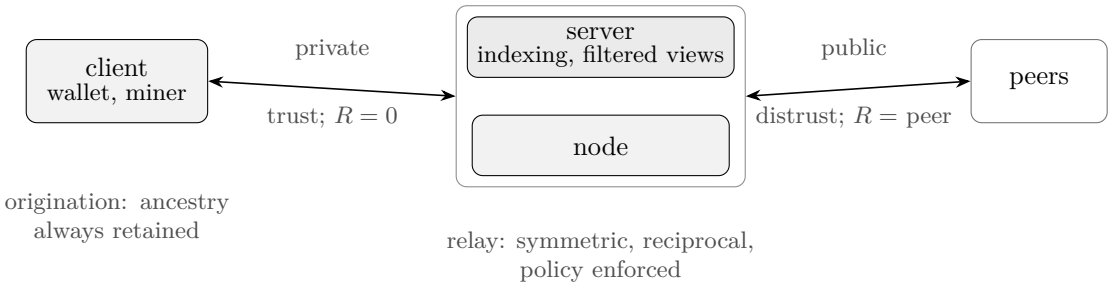


Figure 4: The single distinction. Client-serving features—indexing and filtered views—belong to the server layer, reached over a private channel whose zero fee rate policy is sustained by trust. Peer relay occurs on public channels, where every mechanism of this proposal is a denial-of-service protection warranted by an untrusted counterparty. Because the client can always inject its transactions into its own node, the origination node retains the full unconfirmed ancestry and packaging never falls to the wallet.

the peer protocol: a feature must be reciprocal. The relay "hacks" of Section 1 fail this criterion. BIP037 fails in part by placing service obligations on an untrusted counterparty, but mostly in the other direction: the wallet must trust the node, as the bloom filter discloses the wallet's address set (the privacy leakage that motivated its deprecation). BIP157 fails because it compels a peer to perform work—construction and service of filter indexes—that its counterparty will not reciprocate.

Indexing is a *server* feature. The server is a layer over the node, serving clients—wallets and miners. The client's private channel terminates at this layer; nothing in the peer protocol needs to know that clients exist. Narrowcast relay satisfies the reciprocity criterion by construction: each end of a channel packages for the other against the other's declared policy, with no unreciprocated work and no state retained for a free-riding counterparty.

5.9 Mark Staleness

Marks assert peer state that the peer may silently shed. If a peer evicts a marked transaction under mempool pressure, or a conflicting package ejects it, the sender's marks assert state the peer no longer has; subsequent descendants then relay as orphans and are dropped—correctly, and silently, as there is no reject path. Receipt of a raised `feefilter` may be treated as invalidating a channel's marks on below-rate transactions, covering the rising-floor case cheaply. Overflow eviction is inherently self-announcing—discarding the lowest-feerate entries and raising the announced floor are the same event, so the marks invalidated by the new rate coincide with the state discarded. Only eviction at a constant floor has no signal path. Full coverage requires either an eviction signal or conservative re-packaging upon descendant silence, and is left as a refinement. Confirmation or reconnection resets the channel in any case.

5.10 Features

An orphan is never received. CPFP is inherently enabled by the network, as is replacement by conflict: an acceptable conflicting package relays with no opt-in signal, subsuming BIP125 replacement semantics into conflicting-package fee policy. Wire protocol changes are trivial. No redundant relay of transactions via manually constructed packages occurs. No packaging is re-

quired by a wallet: merely satisfying the feerate of the wallet's peer connection(s) is sufficient, and on the ordinary private channel to the wallet's own node that feerate is zero (Section 5.6). Bumping a stranded transaction requires only sending a new dependent transaction with sufficient aggregate fee. Synchronization may be layered over the set of to-be-relayed packages, just as it would be over the set of to-be-relayed transactions. Nodes obtain the ability to recognize protocol non-compliance given the appearance of orphans.

5.11 Privacy

Marking introduces no disclosure. Marks apply only to a given channel, and the channel peer is already aware of what has been sent to it; the information does not leak across channels.

Privacy is improved in one respect, and the improvement is structural. A node's handling of transactions whose ancestors it lacks is an observable side channel, and it has been used to reconstruct the network graph: TxProbe [17] recovers topology with precision and recall above ninety percent by exploiting the way nodes process out-of-order transactions, relying on injected marker transactions persisting in the orphan pool until they are read back. Topology disclosure of this kind is what enables targeted eclipse attacks and assists in deanonymizing the origin of a transaction. A node that never accepts an orphan, and never requests a missing ancestor, does not exhibit the behavior the technique observes. The channel carries no signal to measure, so the vector is removed rather than mitigated, with no countermeasure to configure, tune, or maintain.

If a node's `feefilter` message value does not match the actual acceptance fee level (such as when the `feefilter` is randomized as suggested in BIP133), the node will be self-orphaning incoming packages.

5.12 Compatibility and Discovery

The proposal is backward compatible. Discovery is by protocol version above a version number to be assigned.

6 Narrowcast Mempool Message

A `mempool` response must conform to a single constraint: it must be an "acceptable" package. This one constraint accomplishes several important objectives. Any transaction sent in a `mempool` response is marked. A peer may therefore issue `mempool` requests iteratively, receiving at most a block's worth of transactions at a time. The transactions are partially ordered, allowing the peer to process the response as it would any package. There are no orphans, allowing it to fully validate the package.

In order to allow for more efficient grouping of message traffic, the minimal-sufficiency constraint is dropped. A node must respond with a sufficient package and should attempt to maximize the package while remaining within block constraints. The BIP035 `inv` size constraint bug is thereby resolved.

Eliminating redundant ancestors given a new connection and a populated mempool is outside the scope of this proposal. This implies mempool synchronization, which is better addressed by a proposal similar to BIP330. Synchronization would complement this proposal by bootstrapping mark state.

7 Silent Implementation

This proposal can be implemented with no change to network protocols. The only difference would be in the issuance of packages with `MSG_TX`/`MSG_WTX` identifiers as opposed to `MSG_PACKAGE`. Existing nodes would process transactions individually and not benefit from the packaging. Supporting nodes would process sets of `MSG_TX`/`MSG_WTX` transactions as packages and may or may not decide to drop nodes that issue packages that are not "acceptable."

A Channel Package Computation

The operational form of the outgoing walk (Section 4.2.2), per Mienk's contemporaneous draft [24] revised in August 2026 [25]. Let $Bytes(t)$, $Sigops(t)$, and $Fee(t)$ be the virtual size, sigop count, and fee of transaction t; b_{max} and s_{max} the block-scope bounds; $R(c)$ the fee rate required by channel c (`feefilter`); $Known_c(t)$ true iff t is marked on channel c (sent to or accepted from the peer, and still in the pool); and $Confirmed(t)$ true iff t is in the blockchain.

The computation runs in two phases. Phase 1 collects the unconfirmed, unmarked ancestry of t in postorder using an explicit stack: postorder guarantees that every in-package ancestor is emitted before its descendant regardless of graph shape, yielding a valid partial order (preorder emission fails when parents spend parents). Phase 2 scans that order greedily, emitting a package at each first crossing of the sufficiency threshold. This is the operational form of the minimal-sufficiency constraint of Section 4.1.2. Exact minimality—the smallest sufficient subset—is a knapsack problem and is in any case unverifiable by the receiver, which cannot see what was withheld. The realizable standard is *prefix-minimality*: no proper ancestor-closed prefix of the emitted package is itself sufficient. It is obtained by construction at the first threshold crossing, and the receiver verifies it in the same linear pass it uses to validate. Every prefix of a topological order is ancestor-closed within the walked set, and members of previously emitted packages are marked at emission, so no announced package is an orphan. An insufficient residual (possibly including t itself) is not announced and not marked; sufficient ancestor packages already emitted remain valid. Sufficiency is tested by cross-multiplication, avoiding division.

A single-element package—t alone, with all ancestors confirmed or marked—reduces to ordinary transaction announcement: Phase 1 yields $[t]$ and Phase 2 announces it iff t meets the peer's rate, the degenerate case of Section 4. Complexity is as analyzed in Section 5.2: each phase is linear in the unmarked unconfirmed ancestry, pruned at marked and confirmed boundaries; marking is $O(1)$ per transaction per channel, at most once per transaction per channel over a channel's lifetime.

References

[1] J. Garzik, "BIP035: mempool message," August 2012 (deployed).

[2] M. Hearn and M. Corallo, "BIP037: Connection Bloom filtering," October 2012 (deployed).

[3] D. A. Harding and P. Todd, "BIP125: Opt-in Full Replace-by-Fee Signaling," December 2015 (deployed).

[4] A. Morcos, "BIP133: feefilter message," February 2016 (deployed).

[5] E. Lombrozo and P. Wuille, "BIP144: Segregated Witness (Peer Services)," January 2016 (deployed).

[6] M. Corallo, "BIP152: Compact Block Relay," April 2016 (deployed).

[7] B. Denby, A. Miller, G. Fanti, S. Bakshi, S. B. Venkatakrishnan, and P. Viswanath, "BIP156: Dandelion—Privacy Enhancing Routing," June 2017 (rejected; closed).

[8] O. Osuntokun, A. Akselrod, and J. Posen, "BIP157: Client Side Block Filtering," May 2017 (deployed).

[9] G. Naumenko and P. Wuille, "BIP330: Transaction announcements reconciliation" (Erlay), September 2019 (draft).

[10] G. Zhao, "BIP331: Ancestor Package Relay," August 2022 (draft).

[11] S. Daftuar, "BIP338: Disable transaction relay message," September 2020 (closed; not deployed).

[12] S. Daftuar, "BIP339: WTXID-based transaction relay," February 2020 (deployed).

[13] S. Daftuar, "Package relay design questions," Bitcoin Core issue 14895, December 2018.

[14] G. Zhao, "Package Relay Proposal," bitcoin-dev mailing list, May 17, 2022.

[15] E. Voskuil, "Packaged Transaction Relay," bitcoin-dev mailing list, June 8, 2022 (thread continued through October 2022).

[16] G. Zhao, "p2p: improve TxOrphanage denial of service bounds," Bitcoin Core pull request 31829, merged July 2025 (release 30.0).

[17] S. Delgado-Segura, S. Bakshi, C. Pérez-Solà, J. Litton, A. Pachulski, A. Miller, and B. Bhattacharjee, "TxProbe: Discovering Bitcoin's Network Topology Using Orphan Transactions," *Financial Cryptography and Data Security*, 2019.

[18] M. A. Imtiaz, D. Starobinski, and A. Trachtenberg, "Characterizing Orphan Transactions in the Bitcoin Network," December 2019.

[19] L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, vol. 21, no. 7, 1978.

[20] A. Demers et al., "Epidemic Algorithms for Replicated Database Maintenance," *Proc. ACM PODC*, 1987.

[21] K. Birman, A. Schiper, and P. Stephenson, "Lightweight Causal and Atomic Group Multicast," *ACM Transactions on Computer Systems*, vol. 9, no. 3, 1991.

[22] A. Carzaniga, D. S. Rosenblum, and A. L. Wolf, "Design and Evaluation of a Wide-Area Event Notification Service," *ACM Transactions on Computer Systems*, vol. 19, no. 3, 2001.

[23] C. Feather, "Network News Transfer Protocol (NNTP)," RFC 3977, October 2006.

[24] P. Mienk, "Iterative Channel Package Computation," libbitcoin-network wiki, 2022.

[25] "Iterative Channel Package Computation: A Corrected Algorithm," revision note, August 2026.

Algorithm 1 PACKAGE(t, c) — run per channel c on acceptance of t, and per channel over unmarked pool leaves on a decrease of $R(c)$.

Phase 1 — collect postorder ancestry
order $\leftarrow [\,]$; *Visited* $\leftarrow \emptyset$; *stack* $\leftarrow [(t, \text{EXPAND})]$
while *stack* not empty **do**
  $(x, \mathit{phase}) \leftarrow \mathit{stack}.\text{pop}()$
  if *phase* = EXPAND **then**
    if $x \in \mathit{Visited}$ **then continue**
    end if
    $\mathit{Visited} \leftarrow \mathit{Visited} \cup \{x\}$
    $\mathit{stack}.\text{push}((x, \text{EMIT}))$
    for all transactions p referenced by inputs of x **do**
      if $p \notin \mathit{Visited} \wedge \neg \mathit{Known}_c(p) \wedge \neg \mathit{Confirmed}(p)$ **then**
        $\mathit{stack}.\text{push}((p, \text{EXPAND}))$
      end if
    end for
  else
    append x to *order* ▷ postorder: ancestors precede descendants
  end if
end while

Phase 2 — greedy prefix-minimal emission
$P \leftarrow [\,]$; $f, b, s \leftarrow 0$
for all x in *order* **do**
  if $b + \mathit{Bytes}(x) > b_{max} \vee s + \mathit{Sigops}(x) > s_{max}$ **then**
    return ▷ excessive: residual not announced, not marked
  end if
  append x to P; $f \mathrel{+}= \mathit{Fee}(x)$; $b \mathrel{+}= \mathit{Bytes}(x)$; $s \mathrel{+}= \mathit{Sigops}(x)$
  if $f \geq R(c) \cdot b$ **then** ▷ first crossing: prefix-minimal
    announce P to c
    for all $y \in P$ **do** $\mathit{Known}_c(y) \leftarrow$ true
    end for
    $P \leftarrow [\,]$; $f, b, s \leftarrow 0$
  end if
end for
▷ insufficient residual P is not announced, not marked